vtslides API

vtslides — lay out whole pages in Typst, `mark` what should morph, and let the browser do the rest.

The identity channel is a **label**: the SVG export turns `<tbl>` into a `<g data-typst-label>` wrapping the content, so the subtree is the boundary; on the PDF side a label costs nothing, so nothing is gated by backend.

Three words. A **transition** is a change of layout state (page to page, frame to frame) and runs on the View Transitions API. An **element animation** is one object varying with a parameter (`mark` with several states, stepped with →) and runs on Web Animations. A **continuous animation** is an object moving on its own, without keys (`slide(anim:)`). The test is one sentence: did the layout change, or is one object moving? Design trade-offs and browser pitfalls are in the README.

vtslides

- mark()
- deck()
- slide()

Variables

- transitions
- anim-defaults

mark

Give a piece of content a name. The same `key` on two adjacent pages makes the browser pair them and interpolate — position, size, colour, rotation. This is a **transition**: the layout changed.

Several bodies make an **element animation**: N states of the same object. While presenting, → moves from one state to the next and ← back along the same path; the page turns only once the states are exhausted. Between states nothing cross-fades — the object's own geometry moves: the browser (Web Animations) interpolates between the corresponding SVG nodes of two states, the path's `d`, transform, colours, stroke width, opacity. So the states must be **the same drawing under different parameters** — same structure, same number of elements, only the numbers differ; write a function `fit1` and feed it a few values of `t`:

```
#mark("wave", ..range(0, 6).map(t => wave(t)))
```

What has no in-between — text that changes, a path against an arc, states whose element count differs — cross-fades. The PDF has no player and shows the last state (the finished figure, like a handout). In the HTML the other states are stacked on the first state's box (`place`, taking no layout space) and hidden by the runtime as data.

If the page's `slide(anim:)` names the same `key`, the states become the keyframes of a continuous animation, played over time instead of stepped; the PDF then shows the first state (the page at rest).

The inner `box` is not decoration: a label attaches to the preceding element, and without the box the thing it would attach to produces no SVG node at all. The layout cost is zero and long text still wraps (measured in the README).

Parameters

```
mark(
  key: str,
  transition: none  str  dictionary,
  ..states: content
) -> content
```

key `str`

The name. Same name on two adjacent pages = one pair. Letters, digits, `_` and `-` (it becomes a CSS `view-transition-name`).

transition `none` or `str` or `dictionary`

This object's own enter/leave effect: a name from `transitions`, the same both ways, or `(enter: name, leave: name)` — `enter` is how it appears, `leave` how it disappears (going back replays either in reverse). It applies only when the mark is one-sided in a transition; a paired mark morphs regardless. `"wipe-up"` reveals from the bottom edge up, `"slide"` pushes in from the right (by its own width), `"zoom"` shrinks into place, `"none"` appears at once. Unset (`none`) folds the mark into the page: within a page it cross-fades with the layout, between pages it pushes, wipes or fades with the whole page. Definition, theorem and proof each revealed from a different edge is three marks with one value each. One object has one effect: given on any occurrence of the `key`, it holds for all of them, and two occurrences may not disagree. Settings ride along in the same dictionary (`duration`, `easing`, `zoom`, `push`; see `transitions`) and apply to this mark alone, whatever pace the page keeps.

Default: `none`

..states `content`

One body is a plain mark; several bodies are the states of the object.

deck

The deck: page size, fonts, and on the HTML side the stylesheet and the runtime.

`#show: deck.with(title: "…")`. One compile gives the PDF, one with `--features html` the HTML; side by side under one name, the toolbar's download link finds the PDF.

Parameters

```
deck(
  title: str,
  width: length,
  height: length,
  pdf: auto  none  str,
  duration: int,
  easing: array,
  transition: none  str  dictionary,
  theme: auto  str,
  fill: color,
  font: array,
  body: content
) -> content
```

title `str`

Document title.

Default: `"vtslides"`

width `length`

Layout width. The PDF page size; in the HTML every frame's `html.frame` is the same size — `slide` reads `page.width` / `page.height`, nothing is hard-coded.

Default: `1280pt`

height `length`

Layout height.

Default: `720pt`

pdf `auto` or `none` or `str`

Target of the toolbar's PDF download link. `auto` = the `.pdf` with the HTML's name, `none` = no button, a string is used as is.

Default: `auto`

duration `int`

Transition duration in milliseconds. Adjustable live with `-` / `=` / `0`.

Default: `700`

easing `array`

Transition easing: the two control points of a cubic Bézier, as in CSS `cubic-bezier(x1, y1, x2, y2)`. `(0, 0, 1, 1)` is linear. Used for page transitions and element-animation steps alike.

Default: `(0.32, 0.72, 0, 1)`

transition `none` or `str` or `dictionary`

The default **page-to-page** transition — how unmarked content enters and leaves when turning to another page: a name from `transitions`, the same both ways, or `(enter: name, leave: name)` — `enter` for the page being entered, `leave` for the page being left. Paired marks morph regardless; one-sided marks without an effect of their own fold into the page. The effect `"none"` switches that side at once while the transition runs (so marks still morph); `none` runs no transition between pages at all. Not used between frames of one page: there the layout stays put and only changes incrementally, so root cross-fades and elements enter and leave by `mark(transition:)`. A single page can override it with `slide(transition:)`. The dictionary may carry settings for this transition as well — `duration`, `easing`, `zoom`, `push`; see `transitions`.

Default: `"fade"`

theme `auto` or `str`

Light/dark theme of the player chrome (toolbar, overview, speaker view). `auto` follows the system, `"dark"` / `"light"` fix it. Chrome only; the layout's colours are Typst's.

Default: `auto`

fill `color`

Layout background. The PDF uses `page(fill:)`; `html.frame` carries no page background, so the same value opens the stylesheet as `--vt-page` and is painted under the slides and thumbnails — identical on both sides, and independent of the chrome theme.

Default: `rgb("#111318")`

font `array`

Font stack, glyph-by-glyph fallback in order. The default carries a CJK family for a reason: with a Latin family alone, CJK text falls back glyph by glyph to whatever system family has the glyph, and weights differ within one line.

Default: `("DejaVu Sans", "Noto Sans CJK SC")`

body `content`

The pages.

slide

One page. On the HTML side a whole-page `html.frame` (hence pixel-identical to the PDF); on the PDF side a page.

Several bodies are **frames of the same page**: navigation walks them one by one, the overview merges them into one thumbnail with dots for the positions. Frames still transition with View Transitions, so "the second frame has one more line" is an element-level interpolation, not a page jump.

```
#slide(title: "Two frames")[first][first + something added]
```

Parameters

```
slide(
  title: content  str  none,
  note: content  str  none,
  transition: none  str  dictionary,
  anim: dictionary,
  ..frames: content
) -> content
```

title `content` or `str` or `none`

Shown only in the thumbnail caption, never in the layout. May be content.

Default: `none`

note `content` or `str` or `none`

Speaker notes. HTML only, placed in the page's `<aside class="vt-note">` (not in the layout, not in the PDF) and read by the desk the deck opens on and by the speaker view (`s`). May be content: paragraphs, lists, emphasis all render.

Default: `none`

transition `none` or `str` or `dictionary`

Overrides `deck(transition:)` for this page: a name from `transitions`, the same both ways, or `(enter: name, leave: name)` — `enter` is how this page comes in when turning to it, `leave` how the page before it goes out — with settings of its own if it wants them (`duration`, `easing`, `zoom`, `push`; see `transitions`). Not used between frames of this page; individual elements enter and leave by `mark(transition:)`. Going back, the page being left decides, so a transition always replays in reverse. `none` takes the deck's.

Default: `none`

anim `dictionary`

Continuous animation, running while the page rests on this frame. `key` = `spec`. The options are `duration` (ms), `delay` (ms), `iterations` (`none` = without end), `direction` (`"normal"`, `"reverse"`, `"alternate"`, `"alternate-reverse"`) and `easing` (a cubic Bézier as in `deck(easing:)`, over each iteration); `anim-defaults` fills in the rest:

```
anim: (spin: (keyframes: ((transform: "rotate(0)"), (transform: "rotate(1turn)")), duration: 4000))
```

Three sources of keyframes: `keyframes` animates the mark itself (transform, opacity, …); `(follow: "track", duration: 3000)` runs the mark's centre along the first path of `mark("track")` on the same frame (a ball on a track), `orient: true` turns it with the tangent; with neither, and the `mark` of that `key` carrying several states, those states are the keyframes — the N poses one piece of code computed, played continuously (double pendulum, waves). Starts after the page transition, pauses when the frame is left.

Default: `(:)`

..frames `content`

The frames of this page. None at all is one blank page.

transitions `array`

Effect names. One is an effect for both sides of a transition; a pair `(enter:, leave:)` names the two sides separately. `"fade"` cross-fades; `"slide"` pushes horizontally (forward, the new page comes in from the right and the old one leaves to the left); `"rise"` pushes vertically (in from the bottom); `"zoom"` scales about the centre (the new one shrinks into place from three times its size, fading in, the old one grows away, fading out) with the screen as the focal plane: whichever is larger than life is nearer than the focus, and a lens a quarter of the stage wide spreads each of its points over a circle that grows with the magnification, so a stroke thinner than the circle casts only a diluted shadow — text arrives as a haze and condenses as it lands; `"wipe-left"` / `"wipe-right"` / `"wipe-up"` / `"wipe-down"` reveal, named by the direction the front travels (`wipe-up` sweeps up from the bottom edge: the new page appears, the old one is covered); `"none"` switches at once. Going back always replays in reverse.

Everywhere a transition is given — `deck`, `slide`, `mark` — the dictionary may also carry settings for that one transition, next to the effects:

```
transition: (effect: "zoom", duration: 400, zoom: 6)
transition: (enter: "slide", leave: "fade", duration: 250, push: -100%)
```

`effect` names one effect for both sides, `enter` and `leave` the two sides separately; every other key is a setting. `duration` is milliseconds, as in `deck(duration:)`; `easing` is four numbers, as in `deck(easing:)`; the rest are the effects' own knobs — `zoom`, how many times life size the zoom starts at, and `push`, how far slide and rise travel (negative sends them the other way). A setting is a variable `deck.css` reads, so the stylesheet is the vocabulary for both halves and a typo is a compile error. The Typst side turns each bundle into one CSS rule and names it after what it holds; the name travels as a view transition type on the page's frames and as a `view-transition-class` on a mark. The presenter's speed keys still divide every duration, whoever wrote it.

anim-defaults

What a continuous animation runs with unless its spec says otherwise: `iterations: none` is without end.